

Introduction à l'apprentissage profond
TP3
S.Boudra & I.Yahiaoui

Partitionnement des données en train/test

Partitionnement manuelle

Après avoir chargé les données, on peut récupérer les données train (`X_train`), leurs étiquettes (`y_train`), ainsi que les données test (`X_test`) et leurs étiquettes (`y_test`) à l'aide de la fonction `train_test_split()`, ou on doit spécifier le pourcentage des données dédiées au test. Ensuite, lors de l'entraînement, on passe cette partition à l'argument `validation_data` de la fonction `fit()`.

```
from sklearn.model_selection import train_test_split
# load diabetes dataset
# split into input (X) and output (Y) variables
# split into 67% for train and 33% for test
X_train, X_test, y_train, y_test = train_test_split(X, Y,
test_size=0.33, random_state=seed)
# create model
# Compile model
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test,y_test),
epochs=150, batch_size=10)
```

Partitionnement automatique

Une autre façon d'obtenir une partition train/test est de définir le pourcentage des données test de l'argument `validation_data` de la fonction `fit()`.

```
# load diabetes dataset
dataset = numpy.loadtxt("diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
# create model
# Compile model
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10)
```

Partitionnement en k-fold et validation croisée

L'évaluation du modèle avec la validation croisée consiste à définir les *kfold* avec la fonction `StratifiedKFold()`, ensuite utiliser la fonction `cross_val_score()` qui va évaluer le

modèle sur k-fold. Le résultat final étant la moyenne des accuracy du modèle sur chacune des partition k-fold de la base de données.

```
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score

# Function to create model, required for KerasClassifier
def create_model():
    # create model
    # Compile model
    return model

# fix random seed for reproducibility
seed = 7
np.random.seed(seed)
# load diabetes dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, epochs=150, batch_size=10,
verbose=0)
# evaluate using 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=seed)
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Exercices

1. Tester les trois méthodes de partitionnement des données sur la base *diabetes*¹ pour la classification binaire.
2. Tester les trois méthodes de partitionnement des données sur la base *seeds*² pour une classification multi-classe.

¹ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>

² <https://archive.ics.uci.edu/ml/datasets/seeds>

Introduction à l'apprentissage profond
TP3
S.Boudra & I.Yahiaoui

Évaluation des paramètres du modèle Keras avec une grille de recherche (Grid Search)

Dans cette partie, on se penche sur l'évaluation des paramètres du modèle Keras avec une grille de recherche afin de trouver la combinaison optimale qui optimise des performances du réseau de neurones.

L'optimisation avec une grille de recherche est définie via la classe `GridSearchCV` de scikit-learn, où un objet de cette classe reçoit les trois attributs suivants :

<code>estimator</code>	Modele keras avec scikit-learn
<code>param_grid</code>	Un <code>dict</code> avec les noms de paramètres (string) comme 'key', et une liste des valeurs à évaluer.
<code>cv</code>	Validation croisée.
<code>n_jobs</code>	-1 Pour paralléliser la grille de recherche

Par la suite, les attributs `best_score_` et `best_params_` permettent de retrouver la valeur optimale du paramètre évalué avec le meilleur scores associé.

```
from sklearn.model_selection import GridSearchCV
# define the grid search parameters
neurons = [1, 5, 15, 25, 30]
param_grid = dict(model__neurons=neurons)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1,
cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_,
grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

En partant du principe de la grille de recherche on va pouvoir évaluer les paramètres suivants d'un réseau de neurones :

1. Méthode d'initialisation des poids du réseau.
2. Les différentes fonctions d'activation.
3. Nombre de neurones dans la couche cachée.
4. Algorithme d'optimisation pour le calcul du gradient stochastique.
5. Taux d'apprentissage et Momentum.
6. Taille du batch et nombre d'epoch.

Les parties suivantes montrent chaque cas d'optimisation des paramètres d'un réseau de neurones.

Méthode d'initialisation des poids du réseau³

```
# Function to create model, required for KerasClassifier
def create_model(init_mode='uniform'):
    # create model
    model = Sequential()
    # Compile model
    return model
# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10,
verbose=0)
# define the grid search parameters
init_mode = ['uniform', 'normal', 'zero']
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```

³ <https://keras.io/initializers/>

Les différentes fonctions d'activation⁴

```
# Function to create model, required for KerasClassifier
def create_model(activation='relu'):
    # create model
    # Compile model
    return model
# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10,
verbose=0)
# define the grid search parameters
activation = ['relu', 'tanh', 'sigmoid', 'linear']
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```

Nombre de neurones dans les couches cachées.

```
# Function to create model, required for KerasClassifier
def create_model(neurons):
    # create model
    model = Sequential()
    # Compile model
    return model
# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10,
verbose=0)
# define the grid search parameters
neurons = [1, 5, 15, 25, 30]
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```

⁴ <https://keras.io/activations/>

Algorithme d'optimisation pour le calcul du gradient stochastique⁵

```
# Function to create model, required for KerasClassifier
def create_model(optimizer='adam'):
    # create model
    model = Sequential()
    # Compile model
    return model

# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10,
verbose=0)
# define the grid search parameters
optimizer = ['SGD', 'RMSprop', 'Adam']
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```

⁵ <https://keras.io/optimizers/>

Taux d'apprentissage et Momentum.

```
# Function to create model, required for KerasClassifier
def create_model():
    # create model
    model = Sequential()
    return model
# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, loss="binary_crossentropy",
optimizer="SGD", epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
learn_rate = [0.01, 0.1, 0.2]
momentum = [0.0, 0.2, 0.4, 0.8]
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```

Taille du batch et nombre d'epoch

```
# Function to create model, required for KerasClassifier
def create_model():
    # create model
    # Compile model
    return model
# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, verbose=0)
# define the grid search parameters
batch_size = [20, 40, 60, 80]
epochs = [10, 50, 100]
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```

Régularisation par dropout

```
# Function to create model, required for KerasClassifier
def create_model(dropout_rate, weight_constraint):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), kernel_initializer='uniform',
    # Compile model
    return model
# load dataset
# split into input (X) and output (Y) variables
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10,
verbose=0)
# define the grid search parameters
weight_constraint = [1.0, 2.0, 3.0]
dropout_rate = [0.0, 0.2, 0.4, 0.6, 0.8]
# grid dictionary
# create the GridSearchCV
# fit the grid
# summarize results
```


Exercices

1. Reprenez les parties de code en les complétant sur la base *diabetes*⁶
2. Construire vers la fin un réseau de neurone avec l'ensemble des paramètres optimaux obtenus lors de la recherche par grille.
3. Evaluer les paramètres d'un réseau de neurones pour la classification multi-classes sur la base *seeds*⁷.
4. Construire le modèle final sur la base *seeds* avec l'ensemble de paramètres optimaux obtenus lors de l'évaluation.

⁶ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>

⁷ <https://archive.ics.uci.edu/ml/datasets/seeds>